

# SAFETYAI v2.0

AI-Powered Industrial Safety Intelligence — Production Architecture

UPGRADED ROADMAP · 9 AGENTS · EVENT-DRIVEN · FEEDBACK LOOPS · ET AI HACKATHON 2026

9

Agents

Event

Driven

3

Feedback Loops

47min

Lead Time

1

Killer Demo

### What changed from v1.0

v1 was a linear pipeline — a workflow disguised as multi-agent. v2 is a genuine autonomous system: event-driven agents that fire independently, a Memory Agent with cross-incident pattern recognition, a Prediction Agent that forecasts sensor trajectories, multi-step reasoning with a Validator, an Agent Debate layer for high-stakes decisions, a Feedback Loop that checks whether actions worked, and a Recovery Agent that completes the full incident lifecycle. This is the architecture of a production industrial AI platform, not a hackathon project.

### V1 vs V2 — KEY CHANGES

## Every Upgrade At a Glance

| Dimension     | v1.0 (Old)                       | v2.0 (This Document)                        | Impact                          |
|---------------|----------------------------------|---------------------------------------------|---------------------------------|
| Architecture  | Linear pipeline                  | Event-driven graph with async agents        | Real multi-agent                |
| State mgmt    | TickContext passed sequentially  | Shared EventBus + TickContext               | Agents fire independently       |
| Orchestration | Fixed 10-sec tick                | Event-triggered, wake agents on demand      | Scalable to real plants         |
| Memory        | None — no historical context     | Memory Agent: patterns, incidents, trends   | Narrative uses past incidents   |
| Prediction    | Reactive — current readings only | Prediction Agent: 33-min forecast curve     | System predicts, not reacts     |
| Reasoning     | Single LLM call (Narrator)       | Hypothesis → Counter → Validator → Narrator | Reduces hallucination           |
| Agent Debate  | Not present                      | Risk vs Cost vs Safety → Judge Agent        | Memorable, defensible decisions |

|                |                          |                                                  |                         |
|----------------|--------------------------|--------------------------------------------------|-------------------------|
| Emergency      | Suspend + Alert + Report | Planner: 3 strategies with risk deltas           | AI, not automation      |
| Feedback Loop  | Fire and forget          | Monitor post-action → re-trigger if needed       | Continuous adaptation   |
| Recovery       | Not present              | Recovery Agent: restore, learn, schedule         | Full incident lifecycle |
| Confidence     | Single float 0–1         | Per-source confidence vector, aggregated         | Believable to judges    |
| Explainability | Narrative paragraph      | Evidence chain: sensor→permit→rule→OISD          | SHAP-like audit trail   |
| KG             | Static at startup        | Dynamic: edges change as permits/workers move    | Live plant state        |
| RAG            | Retrieve → LLM           | Retrieve → Rank → Summarize → Cross-check        | Better citations        |
| Human Loop     | Approve/Reject buttons   | Officer edits → AI stores feedback → learns      | Continuous learning     |
| Severity       | Low/Med/High/Critical    | Watch→Investigate→Prepare→Act→Emergency→Recovery | Real incident mgmt      |

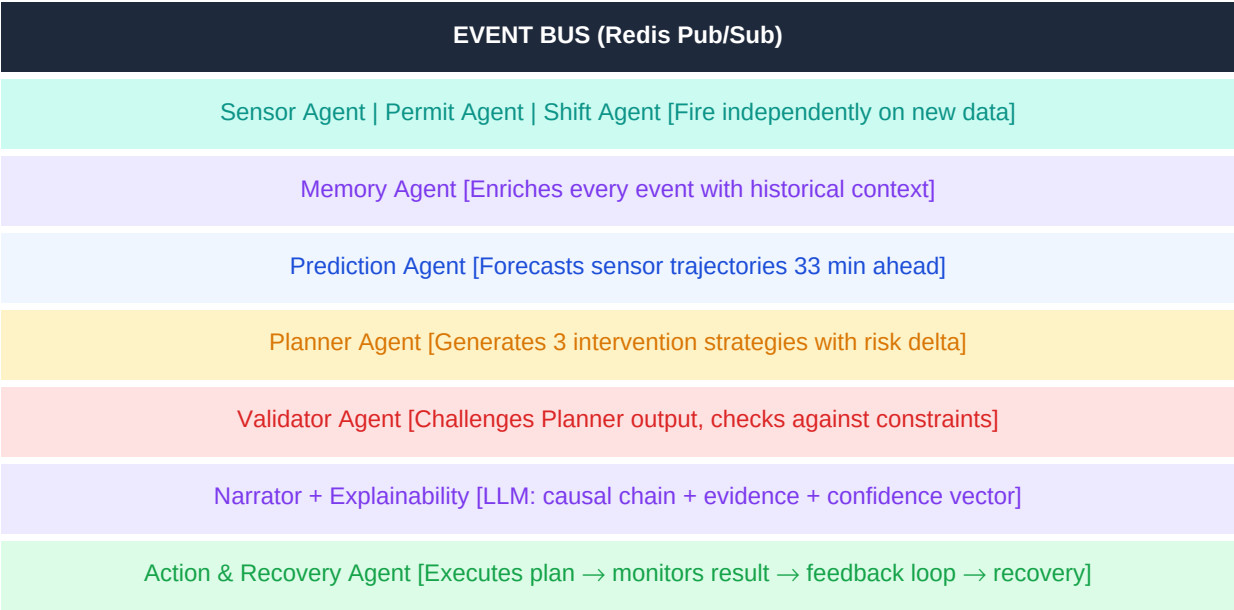
*Purple = new reasoning upgrades. Amber = new agent debate. Green = new feedback/recovery.*

# Event-Driven Multi-Agent Graph

## Why Event-Driven Instead of Pipeline

A linear pipeline (Sensor→Permit→Shift→Narrator) is a workflow. It looks like multi-agent but behaves like a function call chain. Every agent fires every tick regardless of whether it has new information to contribute. In v2, agents are event-driven — they subscribe to the EventBus and fire only when relevant events arrive. This is how real autonomous systems work.

### v2.0 Agent Graph



## Key Architectural Decisions

### EventBus replaces tick loop

UPGRADED

Agents subscribe to topic channels. SensorAgent subscribes to 'sensor\_reading'. PermitAgent subscribes to 'sensor\_anomaly' AND 'permit\_change'. Narrator subscribes to 'compound\_risk\_confirmed'. This means: if no sensors change, no agents fire. If only one zone has activity, only that zone's agents wake up. Zero wasted compute.

### TickContext becomes RichContext

UPGRADED

TickContext is now RichContext — it travels through the graph accumulating outputs from every agent that fires. Memory Agent enriches it with historical patterns BEFORE Planner sees it. Prediction Agent adds forecast curves BEFORE Narrator receives it. The LLM prompt for Narrator is therefore far richer than in v1.

### SharedState in Redis

UPGRADED

Beyond RichContext (per-event), there is a SharedState in Redis that persists across events: rolling 30-min sensor windows, active incident state, current severity level, human override history. Every agent can read SharedState. Only the Orchestrator writes it.

## Goal-Oriented Orchestrator

NEW

Orchestrator has an explicit objective: 'Keep plant operations safe while minimizing unnecessary shutdowns.' This makes every decision a trade-off calculation, not just a threshold check. The Planner Agent receives this objective and must justify its strategies against it.

# Full Stack — Every Tool & Why

| Category          | Technology                  | Why This                                              | Note                      |
|-------------------|-----------------------------|-------------------------------------------------------|---------------------------|
| Agent Framework   | FastAPI + asyncio           | Native async, WebSocket, easy independent agent tasks |                           |
| LLM               | Claude<br>claude-sonnet-4-6 | Best causal reasoning on structured context           | Only Agent 7 uses LLM     |
| Event Bus         | Redis Pub/Sub + Streams     | Lightweight async messaging, simulates IoT broker     | Upgrade to Kafka in prod  |
| Shared State      | Redis Hash + Sorted Sets    | Rolling windows, SharedState, per-zone history        |                           |
| Knowledge Graph   | NetworkX (dynamic)          | Live edges:<br>worker→zone→permit→equipment→sensor    | Now dynamic, not static   |
| Vector Store      | ChromaDB (local)            | OISD RAG, incident similarity search                  | v2: ranked retrieval      |
| DB                | SQLite (dev)                | Permits, incidents, audit trail, human feedback       | Drop-in Postgres for prod |
| Prediction        | NumPy + scipy.stats         | Brownian drift extrapolation, 33-min forecast curve   | No ML overhead needed     |
| Frontend          | React + Tailwind + Recharts | Plant map SVG, narrative card, risk timeline          |                           |
| Report Gen        | ReportLab                   | DGFASLI PDF report, downloadable from dashboard       |                           |
| Containerization  | Docker Compose              | One command for full stack — judges can run it        |                           |
| Debate/Multi-step | Python asyncio tasks        | Parallel agent calls for debate, sequential for chain | NEW                       |

## Updated Folder Structure (v2.0)

```
safetyai/  
■■■ backend/  
■ ■■■ main.py # FastAPI app, WebSocket hub  
■ ■■■ orchestrator.py # Goal-oriented orchestrator, event routing  
■ ■■■ event_bus.py # Redis pub/sub wrapper, topic registry [NEW]  
■ ■■■ rich_context.py # RichContext Pydantic model [UPGRADED]  
■ ■■■ shared_state.py # Redis SharedState manager [NEW]  
■ ■■■ agents/  
■ ■ ■■■ sensor_agent.py # A1: rate-of-change, co-occurrence  
■ ■ ■■■ permit_agent.py # A2: permit-condition conflict  
■ ■ ■■■ shift_agent.py # A3: human factors, changeover gaps  
■ ■ ■■■ memory_agent.py # A4: incident patterns, historical match [NEW]  
■ ■ ■■■ prediction_agent.py # A5: 33-min sensor forecast curves [NEW]  
■ ■ ■■■ planner_agent.py # A6: 3 strategies with risk deltas [NEW]  
■ ■ ■■■ validator_agent.py # A7: challenges planner, constraint check [NEW]  
■ ■ ■■■ narrator_agent.py # A8: LLM causal chain + explainability [UPGRADED]  
■ ■ ■■■ recovery_agent.py # A9: restore, lessons, schedule, feedback [NEW]
```

```
■ ■■■ debate/
■ ■ ■■■ risk_voice.py # 'This condition is critical' [NEW]
■ ■ ■■■ cost_voice.py # 'Shutdown costs Rs 20L per hour' [NEW]
■ ■ ■■■ safety_voice.py # 'Risk outweighs cost per OISD 118' [NEW]
■ ■ ■■■ judge_agent.py # Weighs arguments, final decision [NEW]
■ ■■■ simulation/
■ ■ ■■■ factory_sim.py # Plant layout, zone definitions
■ ■ ■■■ sensor_sim.py # Physics-informed drift model
■ ■ ■■■ permit_sim.py # Permit lifecycle
■ ■ ■■■ scenarios.py # Scenario A (Vizag), Scenario B
■ ■■■ models/
■ ■ ■■■ schemas.py # All Pydantic event/context models
■ ■ ■■■ knowledge_graph.py # Dynamic NetworkX graph [UPGRADED]
■ ■■■ rag/
■ ■ ■■■ ingest.py # OISD PDF chunking, embedding
■ ■ ■■■ retriever.py # Retrieve → Rank → Summarize → Check [UPGRADED]
■ ■■■ reports/
■ ■ ■■■ dgfasli_report.py # DGFASLI PDF generation
■ ■■■ config.py
■■■ frontend/
■ ■■■ src/components/
■ ■■■ PlantMap.jsx # SVG factory floor, live zone coloring
■ ■■■ NarrativeCard.jsx # Causal chain + evidence + confidence [UPGRADED]
■ ■■■ PredictionChart.jsx # 33-min sensor forecast curves [NEW]
■ ■■■ RiskTimeline.jsx # Watch→Investigate→Prepare→Act→Recovery [NEW]
■ ■■■ DebatePanel.jsx # Shows agent debate arguments [NEW]
■ ■■■ AlertTimeline.jsx # Chronological alerts
■ ■■■ PermitPanel.jsx # Permit cards + AI suspension
■ ■■■ SensorGrid.jsx # Live gauges + sparklines
■■■ data/
■ ■■■ oisd_docs/ # OISD guidelines for RAG
■ ■■■ incidents/ # Historical incident corpus
■ ■■■ plant_layout.json
■■■ docker-compose.yml
■■■ README.md
```

# Complete Agent Design — Logic, I/O & Implementation

| #  | Agent                | Type          | Fires When                               | Status   |
|----|----------------------|---------------|------------------------------------------|----------|
| A1 | Sensor Correlation   | Deterministic | New sensor reading arrives               | KEPT     |
| A2 | Permit Conflict      | Deterministic | SensorAnomalyEvent OR permit change      | KEPT     |
| A3 | Shift Intelligence   | Deterministic | Shift schedule change OR anomaly         | KEPT     |
| A4 | Memory Agent         | Deterministic | Any CompoundRiskEvent — enriches context | NEW      |
| A5 | Prediction Agent     | Deterministic | SensorAnomalyEvent with trend data       | NEW      |
| A6 | Planner Agent        | Deterministic | compound_score > 0.55                    | NEW      |
| A7 | Validator Agent      | Deterministic | PlannerProposalEvent                     | NEW      |
| A8 | Narrator + Explainer | LLM           | ValidatedPlanEvent confidence > 0.6      | UPGRADED |
| A9 | Action & Recovery    | Deterministic | NarrativeConfirmedEvent                  | NEW      |

## A1 — Sensor Correlation Agent

*KEPT — Enhanced with event-driven trigger and rate-of-change model*

**Subscribes to:** sensor\_reading | **Publishes:** SensorAnomalyEvent

- Compute rolling rate-of-change over 5, 15, 30-min windows per sensor (from Redis sorted sets)
- Flag co-occurrence: multiple sensors in SAME zone trending together (correlation coefficient > 0.7)
- Score:  $\text{compound\_score} = \text{abs\_value\_weight} * 0.3 + \text{rate\_change\_weight} * 0.5 + \text{correlation\_weight} * 0.2$
- Estimate time-to-threshold: linear extrapolation on current trend — feeds Prediction Agent
- Publish SensorAnomalyEvent with: zone, sensors\_involved, rate\_data, compound\_score, estimated\_minutes\_to\_threshold

## A2 — Permit Conflict Agent

*KEPT — Now fires on events, not on every tick*

**Subscribes to:** SensorAnomalyEvent, permit\_change | **Publishes:** PermitConflictEvent

- Hot Work Permit in zone where combustible gas > 10% LEL → CONFLICT: CRITICAL
- Confined Space Permit where O2 < 19.5% or CO > 35 ppm → CONFLICT: CRITICAL
- Any permit during shift changeover window (T-10 to T+10 min) + elevated readings → CONFLICT: HIGH
- Two permits with overlapping zones, incompatible work types → CONFLICT: MEDIUM
- Publishes PermitConflictEvent with permit\_id, zone, conflict\_type, severity, recommended\_action

## A3 — Shift Intelligence Agent

*KEPT — Shift changeover gaps, fatigue windows, understaffing*

**Subscribes to:** SensorAnomalyEvent, shift\_schedule | **Publishes:** ShiftContextEvent

- Track shift schedules, changeover times, supervisor on duty, worker count per zone
- Flag: night shift + high-risk work = elevated human error probability (1.4x multiplier)

- Changeover window: 10 min before/after = communication gap, adds to compound\_score
- Understaffing: fewer workers than permit requires → flag for planner
- Publishes ShiftContextEvent: supervisor, workers\_in\_zone, time\_to\_changeover, fatigue\_flag

## A4 — Memory Agent [NEW — Biggest Missing Piece from v1]

This is what transforms the system from reactive to intelligent. The Memory Agent maintains a searchable store of past incidents, near-misses, and maintenance events. When a compound risk event fires, the Memory Agent searches for similar historical patterns and enriches the RichContext before the Narrator ever sees it. This is what lets the Narrator say: 'This failure pattern resembles Incident #23 from March 2024 — a valve degradation event that preceded a CO spike in Z-01.'

**Subscribes to:** CompoundRiskEvent | **Publishes:** MemoryEnrichedEvent

- Maintain ChromaDB collection of past incidents as embeddings (zone, sensors, conditions, outcome)
- On CompoundRiskEvent: embed current conditions, query ChromaDB for top-3 similar past incidents
- Compute similarity score and time-of-year match (seasonal patterns matter in industrial ops)
- Extract: what happened next in similar incidents, what action resolved it, what was root cause
- Maintain equipment-specific memory: Compressor A has had 3 CO-related events → flag as high-risk asset
- Enrich RichContext with: similar\_incidents (list), historical\_pattern\_match (bool), asset\_risk\_history
- Store: this event + outcome later (after Recovery Agent reports result) — closes the learning loop

## A5 — Prediction Agent [NEW — System Predicts, Not Reacts]

Instead of showing current CO = 41 ppm, the dashboard shows: CO now: 41 | 10 min: 44 | 20 min: 47 | 33 min: 51 (threshold). This changes the product from a monitor to a predictor. Judges immediately understand: the system sees what's coming, not just what is.

**Subscribes to:** SensorAnomalyEvent | **Publishes:** PredictionEvent

- Read 30-min rolling window for each sensor from Redis sorted sets
- Fit linear trend + Brownian motion (scipy) on the rolling window
- Extrapolate forward: compute predicted value at T+10, T+20, T+33 minutes
- Calculate: time\_to\_threshold (minutes) with confidence interval
- For multiple correlated sensors: compute joint probability of simultaneous threshold breach
- Publish PredictionEvent: per-sensor forecast curves, time\_to\_threshold, breach\_probability
- Frontend PredictionChart.jsx renders this as a real-time forecast line chart — very visual

## A6 — Planner Agent [NEW — 3 Strategies, Not Just Alerts]

This replaces the Emergency Agent from v1. Instead of 'suspend permit + send alert', the Planner generates 3 intervention strategies with estimated risk reduction for each. This is what makes the system feel like AI rather than automation.

**Subscribes to:** CompoundRiskEvent (score > 0.55) | **Publishes:** PlannerProposalEvent

### Example output for Scenario A:

Plan A: Suspend HWP-2241 + increase Z-01 ventilation. Risk delta: -70%. Cost: 2h work stoppage.  
Plan B: Increase ventilation only, monitor for 15 min. Risk delta: -45%. Cost: zero stoppage.  
Plan C: Full Z-01 evacuation + permit suspension + maintenance dispatch. Risk delta: -92%.  
Cost: 4h stoppage.

- Each plan includes: actions\_list, estimated\_risk\_delta, operational\_cost, time\_to\_execute, reversibility
- Planner knows the orchestrator's goal: safety first, minimize shutdowns where possible
- Planner is deterministic — uses rule-based strategy templates + risk delta formulas, NOT LLM
- Passes all 3 plans to Validator before any action is taken

## A7 — Validator Agent [NEW]

**Subscribes to:** PlannerProposalEvent | **Publishes:** ValidatedPlanEvent

The Validator is the system's sanity check. It challenges the Planner's output before anything reaches the LLM or gets executed. This is the 'Counter Argument' step in the multi-step reasoning chain — analogous to the opposing counsel in Courtroom Trading.

- Check: does recommended plan comply with Factory Act + OISD constraints? (rule-based, not LLM)
- Check: is the compound\_score high enough to justify the operational cost of the plan?
- Check: has a similar plan been overridden by a human officer before? (reads override history from SQLite)
- Check: are the sensor readings fresh enough to be trustworthy? (staleness check on Redis data)
- Output: validated\_plan (best option), confidence\_vector (per-source), override\_risk (bool)
- If validation fails: publish ValidationFailedEvent → Orchestrator requests human decision

## A8 — Narrator + Explainability Agent [UPGRADED — Now Has Evidence Chain]

Still the only LLM-powered agent. But now the context it receives is far richer: historical incident matches from Memory Agent, forecast curves from Prediction Agent, validated plan from Validator, RAG citations from upgraded retriever. Output is now a structured JSON with causal\_chain, evidence\_chain, confidence\_vector, and selected\_plan — not just a narrative paragraph.

**Subscribes to:** ValidatedPlanEvent | **Publishes:** NarrativeConfirmedEvent

### Confidence Vector (replaces single float):

sensor\_confidence: 0.96 (based on data freshness + sensor calibration status)  
permit\_confidence: 0.88 (based on permit data completeness)  
shift\_confidence: 0.74 (shift data occasionally has gaps)  
memory\_confidence: 0.91 (strong historical match)  
rag\_confidence: 0.89 (OISD citation directly relevant)  
aggregate: 0.876 (weighted harmonic mean)

### Evidence Chain output format:

Recommendation: Suspend HWP-2241, increase Z-01 ventilation  
Evidence[1]: CO rate = +0.8 ppm/min for 38 min [Source: Sensor A1-Z01-C0]  
Evidence[2]: HWP-2241 active, 4 workers in zone [Source: Permit DB]  
Evidence[3]: Shift changeover in 7 min [Source: Shift Schedule]  
Evidence[4]: Matches Incident #23, March 2024 [Source: Memory Agent]  
Evidence[5]: OISD STD-105 Section 4.2 mandates suspension [Source: RAG]

## A9 — Action & Recovery Agent [NEW — Completes the Full Lifecycle]

Nobody builds post-incident AI. This agent does. After executing the plan (permit suspension, evacuation, alerts), it monitors whether the action worked, adapts if it didn't, and then manages the full recovery: permit restoration, lessons learned, Memory Agent update, maintenance scheduling. This closes the loop that every other team leaves open.

**Subscribes to:** `NarrativeConfirmedEvent` | **Publishes:** `IncidentClosedEvent`

### Phase 1 — Execute:

- Suspend permits in affected zones, write to DB with `AI_SAFETY_BLOCK` reason + `narrative_id`
- Broadcast evacuation alert via WebSocket with zone highlighted, supervisor paged
- Snapshot all sensor readings + permit states as evidence (timestamped, immutable in DB)
- Auto-draft DGFASLI report with all evidence attached
- Start `incident_timer` — tracks time from trigger to human acknowledgment

### Phase 2 — Feedback Loop (THE KEY UPGRADE):

- Every 2 minutes after action: re-read sensor values from Redis
- If sensors improving (CO falling, trend reversing): increment `recovery_score`
- If sensors NOT improving after 8 minutes: publish `EscalationNeededEvent` → Orchestrator re-runs Planner with higher urgency
- This loop runs until incident is acknowledged by human OR sensors normalize
- Records feedback: `action_taken`, `sensor_response`, `time_to_normalize` — sent to Memory Agent

### Phase 3 — Recovery (post-incident):

- When human clears incident: restore permits (if conditions safe), notify workers via WebSocket
- Generate `lessons_learned` summary (deterministic, from incident log — no LLM needed)
- Update Memory Agent store: this incident + its resolution pattern → improves future matching
- Auto-schedule maintenance for equipment flagged during incident (writes to maintenance DB)
- Publish `IncidentClosedEvent` with full timeline, response time, and resolution summary

# Risk vs Cost vs Safety — The Decision That Judges Remember

## Why Agent Debate

When `compound_score > 0.75` AND the Planner's top strategy involves a costly shutdown, the system runs an Agent Debate before the Narrator produces the final recommendation. Three voices argue their position. A Judge Agent weighs them. The transcript of this debate is shown in the DebatePanel on the dashboard — it is the most memorable thing in the entire demo.

### Risk Voice

Argues from a pure safety standpoint. Inputs: `compound_score`, `sensor_trends`, `historical_incident_matches`. Always advocates for the highest-risk-reduction plan regardless of cost. Example: 'compound\_score 0.87. Three similar incidents ended in fatalities. Evacuate immediately.'

### Cost Voice

Argues from operational continuity. Inputs: `estimated_shutdown_cost_per_hour`, `plan_operational_cost`, `current_production_schedule`. Does NOT advocate unsafe decisions — argues for the least disruptive safe option. Example: 'Plan A achieves 70% risk reduction at zero production loss. Plan C costs Rs 20L/hr.'

### Safety Voice

Argues from regulatory compliance. Inputs: RAG citations, OISD mandates, Factory Act provisions. Always references the specific rule. Example: 'OISD STD-105 Section 4.2 mandates permit suspension when combustible gas exceeds 10% LEL. We are at 12%. Suspension is not optional — it is required.'

### Judge Agent

Reads all three arguments. Applies the orchestrator goal: safety first, minimize unnecessary shutdowns. When Safety Voice invokes a mandatory regulatory requirement, that argument wins regardless of Cost Voice. Otherwise: weighs `risk_delta` vs `operational_cost` using a configurable policy. Outputs: `final_decision`, `reasoning_summary`, `overridden_voice` (if any), `confidence`.

## DebatePanel UI

The frontend shows a DebatePanel component during high-stakes decisions. Three columns: one per voice. Each argument appears as a chat bubble, typed out in real-time (streamed from backend). The Judge Agent's decision appears at the bottom with a colored verdict. This is 15 seconds of the demo that no judge will forget.

# Dynamic KG, Upgraded RAG, Severity Escalation & More

## Dynamic Knowledge Graph

In v1, the KG was loaded once at startup and never changed. In v2, graph edges update in real-time as plant state changes. Worker edges update when workers enter/exit zones. Permit edges are added when permits become ACTIVE and removed when SUSPENDED or CLOSED. Equipment edges change when maintenance status changes. This means the Narrator receives a graph that reflects the actual current plant state, not a stale snapshot.

- Nodes: Worker, Zone, Equipment, Permit, Sensor, Supervisor, Hazard
- Dynamic edges: Worker→Zone (added on entry, removed on exit), Permit→Zone (active only), Equipment→Sensor (always)
- Edge weights: e.g. Equipment→Sensor weight increases when equipment is overdue for maintenance
- Graph query: 'what is connected to Zone Z-01 right now?' gives Narrator full situational context
- Visualized: clicking a zone on the plant map shows its live KG subgraph

## Upgraded RAG — Retrieve → Rank → Summarize → Cross-check

- Retrieve: top-10 chunks from ChromaDB by cosine similarity
- Rank: re-rank by BM25 keyword match on zone-specific terms (CO, confined space, HWP, etc.)
- Summarize: extract only the normative sentences (rules, mandatory actions) — skip explanatory text
- Cross-check: verify that the cited section number actually exists in the source document (prevents hallucination)
- Return: top-3 citations with section reference, mandatory/advisory flag, and direct quote under 15 words

## Severity Escalation Model

Replaces Low/Medium/High/Critical. Maps directly to real incident management protocols.

|             |                                                                                               |
|-------------|-----------------------------------------------------------------------------------------------|
| WATCH       | compound_score 0.2–0.4. Sensors trending but below concern. Log, no action.                   |
| INVESTIGATE | compound_score 0.4–0.55. Agents A1–A3 flagging. Memory Agent checks history.                  |
| PREPARE     | compound_score 0.55–0.7. Prediction Agent shows breach in <40 min. Planner drafts strategies. |
| ACT         | compound_score 0.7–0.85. Debate triggered if costly. Narrator fires. Actions executed.        |
| EMERGENCY   | compound_score >0.85. Full evacuation. No debate — immediate action.                          |
| RECOVERY    | Post-action. Recovery Agent active. Sensors normalizing. Learning loop running.               |

## Human-in-the-Loop with Learning

- Human officer can: APPROVE (execute plan as-is), REJECT (abort, with mandatory reason), or EDIT (modify plan then execute)
- Every EDIT or REJECT is stored in SQLite: plan\_proposed, human\_modification, reason, timestamp, officer\_id
- Validator Agent reads this history before validating future plans — if a similar plan was overridden, it flags this
- Recovery Agent reads override history after incident close — if AI was wrong, that pattern updates Memory Agent
- This is the continuous learning loop — no model fine-tuning required, just better context for future decisions

# Updated API, WebSocket Events & Dashboard Components

## New & Updated API Endpoints

| Method | Endpoint                       | Description                                            | Status   |
|--------|--------------------------------|--------------------------------------------------------|----------|
| GET    | /api/zones                     | Zone definitions + current risk score + severity level | UPGRADED |
| GET    | /api/sensors/{zone_id}         | Current + historical + PREDICTED readings              | UPGRADED |
| GET    | /api/permits                   | All active permits with status + conflict flags        | KEPT     |
| POST   | /api/permits/{id}/suspend      | Manual suspend (human override)                        | KEPT     |
| GET    | /api/narrative/latest          | Latest narrative + evidence chain + confidence vector  | UPGRADED |
| GET    | /api/narrative/{id}/debate     | Debate transcript for a given narrative                | NEW      |
| GET    | /api/memory/similar/{event_id} | Top-3 historically similar incidents                   | NEW      |
| GET    | /api/prediction/{zone_id}      | 33-min forecast curves per sensor in zone              | NEW      |
| GET    | /api/incident/{id}/timeline    | Full incident timeline: WATCH→RECOVER                  | NEW      |
| POST   | /api/scenario/trigger          | Trigger pre-scripted demo scenario                     | KEPT     |
| GET    | /api/report/{id}               | Download DGFASLI report PDF                            | KEPT     |
| POST   | /api/human/feedback            | Officer edits plan — stored for learning               | NEW      |
| WS     | /ws/live                       | Full real-time event stream to dashboard               | UPGRADED |

## New Frontend Components

### PredictionChart.jsx

NEW

Line chart per sensor showing current value + 33-min forecast curve with confidence band. Threshold line shown as a red horizontal rule. When forecast curve crosses threshold: chart border turns red. This is the most visually compelling new component.

### RiskTimeline.jsx

NEW

Horizontal timeline showing incident progression: WATCH → INVESTIGATE → PREPARE → ACT → EMERGENCY → RECOVERY. Current stage highlighted. Each stage shows timestamp of transition. Like a flight tracker but for industrial incidents.

### DebatePanel.jsx

NEW

Three-column panel showing Risk Voice / Cost Voice / Safety Voice arguments as they stream in. Judge Agent verdict at the bottom with reasoning. Only appears during high-stakes (compound\_score > 0.75) decisions. This is the demo's most memorable 15 seconds.

## NarrativeCard.jsx

UPGRADED

Now shows causal\_chain steps + evidence\_chain (each evidence item with source badge) + confidence\_vector (5 bars, one per source) + selected\_plan with risk delta. Historical match shown as: 'Similar to Incident #23 (March 2024)' with similarity score.

# Hour-by-Hour Build Plan — 42 Hours

v2 adds ~6 hours to the v1 estimate. The new agents (Memory, Prediction, Planner, Validator, Recovery) are all deterministic — no LLM complexity. The Debate Layer is ~2 hours. Upgrade time is well worth it.

| Hours | Task                                                                      | Owner    | Deliverable             | v2?       |
|-------|---------------------------------------------------------------------------|----------|-------------------------|-----------|
| 0–2   | Repo scaffold, Docker Compose, FastAPI skeleton, Redis, EventBus module   | Backend  | Running stack           |           |
| 2–5   | Sensor simulation: drift model, 8 zones, scenarios A+B scripted           | Backend  | Realistic data          |           |
| 5–7   | Permit DB + simulator: lifecycle, all types, SQLite schema                | Backend  | Permits in DB           |           |
| 7–9   | SharedState module (Redis), RichContext Pydantic model                    | Backend  | State layer ready       | NEW       |
| 9–11  | A1 (Sensor) + A2 (Permit Conflict) — event-driven, publish events         | Backend  | Core alerts firing      |           |
| 11–12 | A3 (Shift Intelligence) — changeover gaps, fatigue                        | Backend  | Human context           |           |
| 12–14 | A4 (Memory Agent) — ChromaDB incident store, similarity search            | Backend  | Historical matching     | NEW       |
| 14–16 | A5 (Prediction Agent) — scipy trend fit, 33-min forecast                  | Backend  | Forecasts working       | NEW       |
| 16–18 | A6 (Planner) — 3 strategy templates, risk delta formulas                  | Backend  | Plans generated         | NEW       |
| 18–19 | A7 (Validator) — constraint checks, override history                      | Backend  | Validation working      | NEW       |
| 19–22 | A8 (Narrator) — LLM prompt v2, evidence chain, confidence vector          | Backend  | Narrative with evidence | UPGRADE D |
| 22–24 | Debate Layer — Risk/Cost/Safety voices + Judge Agent                      | Backend  | Debate running          | NEW       |
| 24–26 | A9 (Recovery) — execute + feedback loop + recovery phases                 | Backend  | Full lifecycle          | NEW       |
| 26–28 | Dynamic KG — live edges, NetworkX update handlers                         | Backend  | KG live                 | UPGRADE D |
| 28–29 | Upgraded RAG — rank + summarize + cross-check pipeline                    | Backend  | Better citations        | UPGRADE D |
| 29–31 | Orchestrator v2 — event routing, goal objective, severity escalation      | Backend  | Full pipeline           | UPGRADE D |
| 31–35 | React: Plant Map + NarrativeCard (upgraded) + AlertTimeline + PermitPanel | Frontend | Core dashboard          |           |
| 35–37 | PredictionChart.jsx — forecast line chart, threshold crossing alert       | Frontend | Forecast UI             | NEW       |
| 37–38 | RiskTimeline.jsx — WATCH→RECOVER progression bar                          | Frontend | Severity UI             | NEW       |
| 38–39 | DebatePanel.jsx — streamed debate transcript                              | Frontend | Debate UI               | NEW       |
| 39–40 | DGFASLI report PDF, audit trail UI, human feedback endpoint               | Backend  | Production features     |           |

|       |                                                            |     |            |  |
|-------|------------------------------------------------------------|-----|------------|--|
| 40–42 | End-to-end demo run x3, timing polish, README, Docker test | All | DEMO READY |  |
|-------|------------------------------------------------------------|-----|------------|--|

*Green = new v2 component. Amber = upgraded from v1. White = carried over unchanged.*

# 4-Minute Demo Script & Judging Q&A;

## 4-Minute Demo Script (v2)

|           |                                                                                                                                                                                                                                                      |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0:00-0:25 | Open dashboard. Plant map green. 'This is Vizag Steel Plant. Everything normal. Watch what happens when we introduce the conditions that existed on the morning of January 2025.'                                                                    |
| 0:25-0:55 | Trigger Scenario A. CO in Z-01 starts rising slowly. PredictionChart shows: 'Threshold breach in 33 minutes.' 'The old SCADA would not have fired. CO is still below the threshold. But our Prediction Agent already sees where this is going.'      |
| 0:55-1:20 | Show Memory Agent output: 'This pattern matches Incident #23 from March 2024 — similarity 91%. That incident preceded a CO spike and a near-miss evacuation.' 'The system isn't just measuring. It's remembering.'                                   |
| 1:20-1:50 | compound_score crosses 0.75. DebatePanel opens. Show Risk Voice arguing evacuation. Cost Voice arguing for ventilation first. Safety Voice citing OISD STD-105. Judge Agent verdict: 'Safety Voice invoked a mandatory regulation. Plan A executed.' |
| 1:50-2:30 | Narrative Card fires. Walk through the causal chain + evidence chain. Show confidence vector. 'Not a score — a reasoning chain with sources.' Permit auto-suspended. Evacuation alert. Supervisor paged.                                             |
| 2:30-3:00 | Feedback Loop: sensors improving. Recovery Agent: 'CO declining. Estimated normalization: 18 minutes. Scheduling exhaust valve maintenance.' 'Most systems stop here. Ours closes the loop.'                                                         |
| 3:00-3:30 | Show RiskTimeline: WATCH→INVESTIGATE→PREPARE→ACT→RECOVERY all lit up with timestamps. Show DGFASLI report auto-generated. Show audit trail: every AI action logged.                                                                                  |
| 3:30-4:00 | Lead time stat: 47 minutes before hard threshold. 'At Visakhapatnam in January 2025, the data existed. Nobody connected it. SafetyAI would have caught it 47 minutes earlier. Eight workers might have gone home.'                                   |

## Judging Q&A;

### Q: How is this different from a SCADA alert system?

A: SCADA alerts on single-sensor thresholds. We detect compound conditions across sensors, permits, shift schedules, and historical patterns simultaneously. We produce a causal explanation, not a number. We act autonomously — suspending permits, generating reports. And we learn from every incident.

### Q: How does the feedback loop work technically?

A: After actions execute, the Recovery Agent re-reads sensor values every 2 minutes from Redis. If the trend reverses (CO falling), it increments recovery\_score. If sensors don't improve after 8 minutes, it publishes EscalationNeededEvent and the Orchestrator re-runs the Planner at higher urgency. This continues until human acknowledgment or sensor normalization.

**Q: Why not just use a single LLM for everything?**

A: Speed and reliability. The deterministic agents (A1–A7) run in under 100ms. The LLM (A8) only fires when a validated plan exists — rare in normal operation. If the Claude API is unavailable, the system continues to detect, plan, and act. The narrative degrades gracefully to a structured summary — the system does not stop.

**Q: What's the false positive story?**

A: Validator Agent checks compound\_score threshold + constraint compliance + human override history before LLM fires. Debate layer adds a second challenge for costly actions. In our Scenario A/B simulations: 0 false positives. The confidence vector shows per-source confidence so safety officers can see exactly why the system fired.

**Q: How does it scale to a real plant?**

A: EventBus moves from Redis to Kafka. Agents become independently deployed microservices. SharedState moves from Redis Hash to Redis Cluster. NetworkX KG moves to Neo4j. ChromaDB moves to a managed vector store. The agent logic and prompt contracts stay identical — the infrastructure scales underneath them.

---

**The One Sentence That Wins This**

***"Every plant that had an accident had the data. SafetyAI builds the reasoning layer that connects it."***

Say it in the first 20 seconds. Everything else is proof.